Dev Workflow One-Sheet

Your team has 10 days to go from validated idea to working MVP. This doc covers the workflow, tools, and rules to get there without chaos.

The Rule

Nobody deploys code they can't explain.

If AI wrote it and you can't walk someone through how it works, it doesn't ship. This is the single rule that prevents your team from accumulating code nobody understands.

The Workflow

Every feature follows this loop. Don't skip steps.

```
1. BRIEF        Define what you're building and why          [write it down]
2. ARCHITECTURE  Decide how it fits into the system          [write it down]
3. PLAN          Break it into phases with tests             [write it down]
4. IMPLEMENT     Build it (AI writes most code here)         [TDD: test first]
5. REVIEW        Read the code. Understand it. Question it.  [another human]
6. MERGE         PR → main                                   [after review]
```

Steps 1-3 are where you think. Step 4 is where AI contributes most. Steps 5-6 are where you catch problems. If you skip 1-3, step 4 produces garbage faster.

You already have `/feature-forge` which walks you through this. Use it.

CLAUDE.md — Your Team's Brain

Create a `CLAUDE.md` file in the root of your repo. This is persistent context that Claude reads at the start of every session. It makes AI assistance dramatically better.

Sample CLAUDE.md

```
# [Your Project Name]

## What This Is
- See docs/ongoing/BRIEF.md

## Tech Stack
- See docs/ongoing/ARCHITECTURE.md

## Project Structure
- `src/` or `app/` — application code
- `tests/` — test files mirror source structure
- `docs/ongoing/` — living docs (product brief, brand position, style guide)
- `docs/features/` — feature briefs, architectures, plans

## Onboarding Docs (docs/ongoing/)
```

```
Always check these docs at the start of a session as onboarding context.

- `PRODUCT_BRIEF.md` — problem, target user, value prop, key features
- `BRAND_POSITION.md` — positioning statement, messaging, tone of voice
- `STYLE_GUIDE.md` — color palette (hex codes), typography, visual tone
- `ARCHITECTURE.md` — system overview, data models, API design (add during build)

These are living documents. Update them as the product evolves.

## Documentation Style
Use Mermaid diagrams for flows, sequences, state machines, data relationships, and decision trees. Diagrams are

## Code Conventions
- [Naming: camelCase for JS, snake_case for Python/Ruby]
- [Component pattern: functional components with hooks]
- [API pattern: RESTful, JSON responses]
- [Error format: { error: string, details?: object }]

## Git Workflow
- `main` is protected. Never push directly.
- Create feature branches: `feature/short-description`
- PRs require one reviewer before merge.
- Commit messages: imperative tense ("Add login flow", not "Added login flow")

## Testing
- Run tests: `[npm test / pytest / bundle exec rspec]`
- Write tests for new features before or alongside code
- Tests live in `[tests/ / spec/ / __tests__/]`

## Environment
- See docs/ongoing/ARCHITECTURE.md

## Known Issues / Gotchas
- [Add things here as you discover them]
- [e.g., "The auth middleware must run before any API route"]
- [e.g., "Don't use library X for Y — it doesn't support Z"]

## Team
- [Name] — [what they're working on]
- [Name] — [what they're working on]
```

Keep it tight. The root CLAUDE.md should stay under ~60 lines of the most important context. The project CLAUDE.md can be much larger (up to ~40k tokens) as your project grows. Update it continuously. When Claude does something wrong, add a note so it doesn't happen again.

Bulletin Board — Agent-to-Agent Communication

When multiple people are coding with AI agents on the same repo, you need a way for agents to leave messages for each other. Create a `BULLETIN.md` file in your repo root with a simple markdown table:

```
# Bulletin Board

| Date | Time | From | Message |
|------|------|------|---------|
| 2026-03-03 | 14:20 | @Alice | Built the auth flow. @Bob: the `/api/auth` endpoint is ready for you to integrat
| 2026-03-03 | 15:45 | @Bob | Connected login form to auth endpoint. Signup flow still needs the email verificat
```

How It Works

Add this to your CLAUDE.md so every agent knows the protocol:

```
## Bulletin Board
- **After every `git pull`**, re-read `BULLETIN.md` for new messages @-mentioning your owner.
- **At session end**, add a row with: date, time, your owner's name, and a message covering (1) what you accompl
- **Use @-mentions** to call out specific teammates when handing off work or requesting something.
- **Cross-domain requests:** If your work touches another teammate's area, leave a message instead of silently m
```

Why This Matters

With 2-4 people each running their own Claude sessions, agents don't share memory. Agent A finishes the API; Agent B starts the frontend 20 minutes later with no idea the API is ready. The bulletin board solves this: every agent checks it on startup and posts to it on shutdown. It's the cheapest coordination tool you'll ever use.

Git for Teams

You have 2-4 people coding. Without basic git hygiene, you will lose work.

Setup

```
# One person creates the repo and pushes initial code
git init
git add .
git commit -m "Initial project setup"
git remote add origin <your-github-url>
git push -u origin main

# Everyone else clones
git clone <your-github-url>
```

Daily Workflow

```
# Start of work: get latest
git pull origin main

# Create a branch for your feature
git checkout -b feature/user-login

# Work, commit often
git add .
git commit -m "Add login form component"

# When ready: push and open a PR
git push -u origin feature/user-login
# Go to GitHub → Pull Requests → New Pull Request
```

Rules

1. **Never push to main directly.** Always use a branch + PR.
2. **One feature per branch.** Don't mix unrelated changes.
3. **Pull before you branch.** Start from the latest main.
4. **Commit often.** Small commits are easy to review and easy to undo.
5. **If you get a merge conflict**, don't panic. Read what changed. Ask a teammate. Claude can help resolve it.

Testing Your MVP

Tests aren't optional. They save you from shipping broken features and from AI-generated code that looks right but doesn't work. You don't need to write test code by hand; Claude will generate tests for you. What you need to know is **what kinds of tests to ask for**.

The TDD Loop

```
RED:      Write a test that describes what should happen. Run it. It fails.
GREEN:    Write the simplest code that makes the test pass.
REFACTOR: Clean up. All tests still pass.
COMMIT.
```

Types of Tests (Know When to Use Each)

Type	What It Tests	When to Use	Tools
Unit tests	Individual functions, components, or modules in isolation	Business logic, data transformations, utility functions	Jest, Vitest, pytest, RSpec
Integration tests	Multiple parts working together (API + database, component + state)	API endpoints, database queries, form submissions	Same as unit + test databases
E2E tests (Playwright)	Full user flows in a real browser	Your core flow: user arrives → does the thing → gets value	Playwright, Cypress
Browser automation (Claude)	Visual verification, clicking through your app	Quick smoke tests, checking responsive design, validating UX	Claude Code's built-in browser tools

What to Test First

1. **Unit tests for your core logic.** Whatever your product actually *does* (matching, calculating, filtering, generating), test that logic in isolation.
2. **Integration tests for your API.** If you have endpoints, test that they return the right data and handle errors.
3. **One E2E test for your core flow.** The path from "user arrives" to "user gets value." If that flow works, your demo works.

Don't write tests for everything. Focus on the path that matters for your demo. Tell Claude what you want to test and it will generate the test code.

Quick Reference

What	How
Start a feature	`/feature-forge` or write a brief manually
Set up AI context	Create `CLAUDE.md` in your repo root
Add onboarding docs	Copy PRODUCT_BRIEF, BRAND_POSITION, STYLE_GUIDE to `docs/ongoing/`

What	How
Create a branch	`git checkout -b feature/short-name`
Open a PR	Push branch, go to GitHub, click "New Pull Request"
Run unit/integration	`npm test` / `pytest` / your framework's test command
Run E2E tests	`npx playwright test`
Get unstuck	Ask Claude, but understand the answer before using it

---

MVP Scoping Checklist

Before you start building, answer these:

- [ ] What is the **one core flow**? (user → action → value)
- [ ] Can you **demo it in 3 minutes**? If not, cut scope.
- [ ] What can you **fake**? (Manual processes behind the scenes, mock data, hardcoded responses)
- [ ] Who on your team is **building what**?
- [ ] Do you have a `CLAUDE.md` in your repo?
- [ ] Is your git workflow set up? (branches, PRs, reviewer assigned)